

Software Design Document (SDD)

Module	I2C Driver
Version	1.0.0
Team	Carlos Cayetano · Adrian Amaro · Adalberto Aguirre · Daniel López
Date	26/05/2026
Institution	CETYS Universidad

1. Introduction

1.1 Purpose

This Software Design Document (SDD) describes the internal architecture and design of the I2C driver module developed for the STM32F411RE microcontroller, conforming to IEEE Std 1016-2009. The document covers data structures, register-level algorithms, control flow, and the complete public API. Its purpose is to guide implementation and testing of any software module that depends on I2C communication.

1.2 Scope

The I2C driver is a bare-metal, low-level firmware component for the STM32F411RE Nucleo board. It configures I2C1 in standard mode at 100 kHz using APB1 at 16 MHz (HSI default), with PB8 as SCL and PB9 as SDA, both configured as AF4 open-drain outputs. The driver exposes a four-function API for writing and reading single or multi-byte payloads with or without a register address pointer. In Lab Assignment 05, this driver is used by the ADXL345 sensor module to read accelerometer data over I2C. External 4.7 kΩ pull-up resistors to 3.3 V are required on both lines; the internal STM32 pull-ups (~40 kΩ) are insufficient for reliable 100 kHz operation.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
ACK	Acknowledge — a single low bit sent by the receiver after each byte to confirm receipt.
AF	Acknowledge Failure — SR1 bit 10; set when no ACK is received from the slave device.
ADDR	Address Sent flag — SR1 bit 1; set by hardware after the address byte has been transmitted and ACKed.
APB1	Advanced Peripheral Bus 1 — the bus that clocks I2C1 at 16 MHz on the STM32F411RE (default HSI).
BTF	Byte Transfer Finished flag — SR1 bit 2; set when both the data register and shift register have been fully transmitted.
CCR	Clock Control Register — programs the I2C clock divider; $CCR = f_{APB1} / (2 \times f_{SCL})$.
CMSIS	Cortex Microcontroller Software Interface Standard — vendor C headers mapping to hardware register addresses.
CR1	I2C Control Register 1 — contains PE (peripheral enable), START, STOP, ACK, and SWRST bits.
CR2	I2C Control Register 2 — contains the FREQ field specifying APB1 frequency in MHz.

DR	I2C Data Register — the transmit/receive holding register; writing queues a byte for transmission, reading retrieves a received byte.
FR	Functional Requirement — a numbered requirement from the SRS that the driver must satisfy.
HSI	High-Speed Internal oscillator — the 16 MHz default clock source on the STM32F411RE after reset.
I2C	Inter-Integrated Circuit — a two-wire synchronous serial protocol using SCL (clock) and SDA (data).
I2C1	The I2C peripheral instance used by this driver, mapped to PB8 (SCL) and PB9 (SDA).
NACK	Not Acknowledge — no ACK pulse from receiver; signals the final byte in a read sequence.
NFR	Non-Functional Requirement — a quality attribute requirement (e.g., no external library dependencies).
PB8/PB9	GPIO pins B8 and B9 on the STM32F411RE, assigned to I2C1_SCL and I2C1_SDA respectively.
PE	Peripheral Enable bit — CR1 bit 0; activates the I2C peripheral after configuration.
RESTART	Repeated START — a new START condition generated without a preceding STOP; used to switch direction in a read transaction.
SB	Start Bit flag — SR1 bit 0; set by hardware after a START condition has been generated.
SCL	Serial Clock Line — the I2C clock wire, driven by the master.
SDA	Serial Data Line — the I2C data wire, bidirectional.
SR1/SR2	I2C Status Registers 1 and 2 — contain event and error flags. Reading SR1 followed by SR2 clears the ADDR flag.
SRS	Software Requirements Specification — the requirements document this SDD traces to.
SWRST	Software Reset bit — CR1 bit 15; forces the I2C peripheral into a reset state to recover from bus errors.
TRISE	Rise Time Register — programs the maximum SCL/SDA rise time in units of 1/fAPB1; TRISE = 1000 ns / tAPB1 + 1.

1.4 Requirements

All functional requirements for this module are defined in SRS-I2C_Driver. The table below traces each requirement to the driver function that satisfies it.

Req. ID	Description
FR-1	The driver shall configure I2C1 clock, GPIO pins PB8/PB9 as AF4 open-drain, CCR, and TRISE for standard mode 100 kHz. Implemented by i2c_init().
FR-2	The driver shall set I2C1 to standard mode (100 kHz) using a 16 MHz APB1 source. Implemented by i2c_init().
FR-3	The driver shall generate a START condition and transmit the device address with write intent, followed by a register address byte. Implemented by i2c_writeRegDevice().
FR-4	The driver shall transmit len data bytes to the register address, then generate a STOP condition. Implemented by i2c_writeRegDevice().
FR-5	The driver shall generate a START condition and transmit the device address with write intent, without a register address. Implemented by i2c_writeDevice().
FR-6	The driver shall transmit len data bytes directly to the device, then generate a STOP condition. Implemented by i2c_writeDevice().
FR-7	The driver shall perform a two-phase read: write phase sends register address, then a RESTART switches to read direction. Implemented by i2c_readRegDevice().
FR-8	The driver shall read len bytes, sending NACK on the final byte, then generate a STOP condition. Implemented by i2c_readRegDevice().
FR-9	The driver shall generate a START condition and transmit the device address with read intent, without prior register pointer write. Implemented by i2c_readDevice().
FR-10	The driver shall read len bytes from the device with NACK on the final byte and a STOP condition. Implemented by i2c_readDevice().

2. System Architecture

2.1 High-Level Design

The I2C driver is positioned in the communication layer of a four-tier embedded software stack. The application layer (`main.c`) operates at the highest level of abstraction, calling sensor functions without any knowledge of the underlying bus protocol. The sensor layer (`sensor.c`, implementing the ADXL345 driver) translates logical operations—such as verifying the device ID or reading a burst of acceleration data—into I2C read/write transactions. The I2C driver layer handles all bus-level operations: generating START and STOP conditions, managing acknowledgement, and polling status flags. Below that, CMSIS provides the C register-map structures that map directly to I2C1 hardware addresses.

- Application Layer: `main.c` calls `sensor_init()` and `sensor_readData()` without any direct I2C interaction.
- Sensor Layer (`sensor.c`): maps ADXL345 register access to `i2c_writeRegDevice()` and `i2c_readRegDevice()` calls.
- I2C Driver Layer (`i2c.c`): manages START/STOP generation, address transmission, byte-level read/write, and timeout error recovery.
- Hardware Layer: STM32F411RE I2C1 peripheral registers, accessed through the CMSIS `I2C_TypeDef` structure.

2.2 Module Interfaces

The public API is declared in `i2c.h` and consists of five functions grouped into two categories: initialization (`i2c_init`) and data transfer (`i2c_writeRegDevice`, `i2c_writeDevice`, `i2c_readRegDevice`, `i2c_readDevice`). The driver depends on `gpio_driver.h` for `gpio_initPort(GPIOB)` and `gpio_setAlternateFunction(GPIOB, pin, 4)`. After the GPIO configuration, all further hardware access goes directly through the CMSIS I2C1 register structure.

Key CMSIS registers used: I2C1->CR1 (PE, START, STOP, ACK, SWRST), I2C1->CR2 (FREQ field = 16), I2C1->CCR (= 80 for 100 kHz), I2C1->TRISE (= 17), I2C1->SR1 (SB, ADDR, BTF, TXE, RXNE, AF flags), I2C1->SR2 (BUSY flag), I2C1->DR (data register), RCC->APB1ENR (I2C1EN bit), GPIOB->OTYPER (open-drain), GPIOB->PUPDR (no pull-up), GPIOB->OSPEEDR (high speed).

3. Detailed Design

3.1 Class Diagram

Figure — Figure 1. Class Diagram — I2C Driver

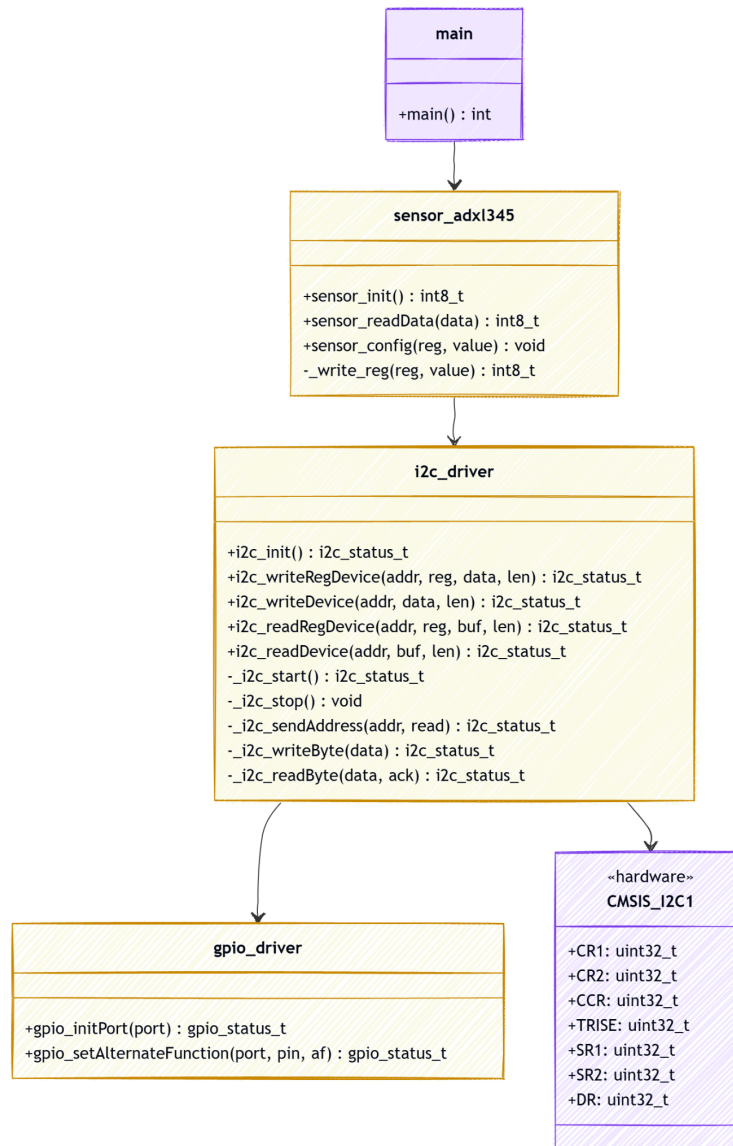


Figure 1. Class Diagram — I2C Driver

3.2 Module Behavior

The following eight figures document the I2C driver behavior. Figures 2 and 3 cover the initialization sequence. Figures 4 and 5 document the `i2c_writeRegDevice` function, which is the most commonly used write path. Figures 6 and 7 cover `i2c_readRegDevice`, the two-phase read with repeated START. Figure 8 is the driver state machine.

Figure — Figure 2. Sequence Diagram — `i2c_init`

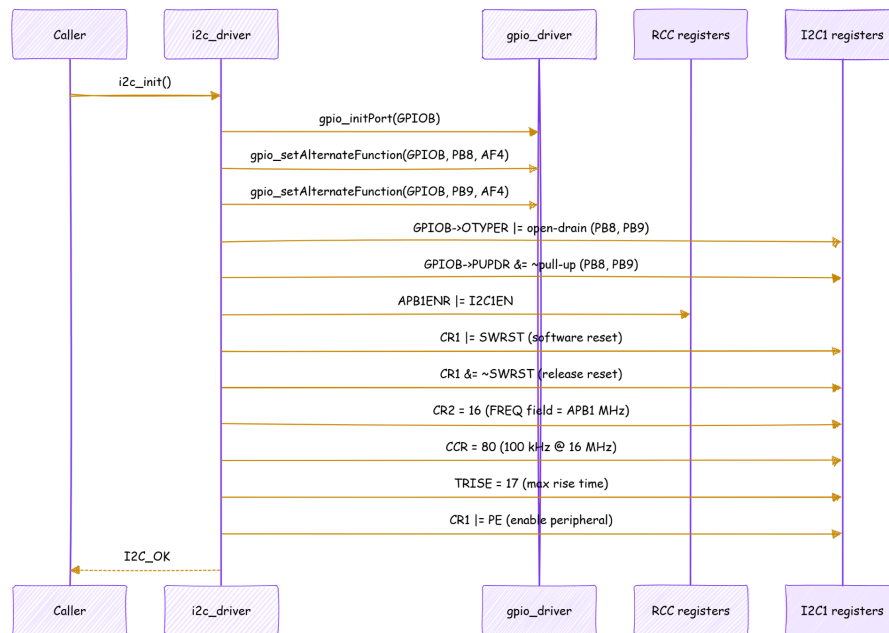


Figure 2. Sequence Diagram — `i2c_init`

Figure — Figure 3. Activity Diagram — `i2c_init`

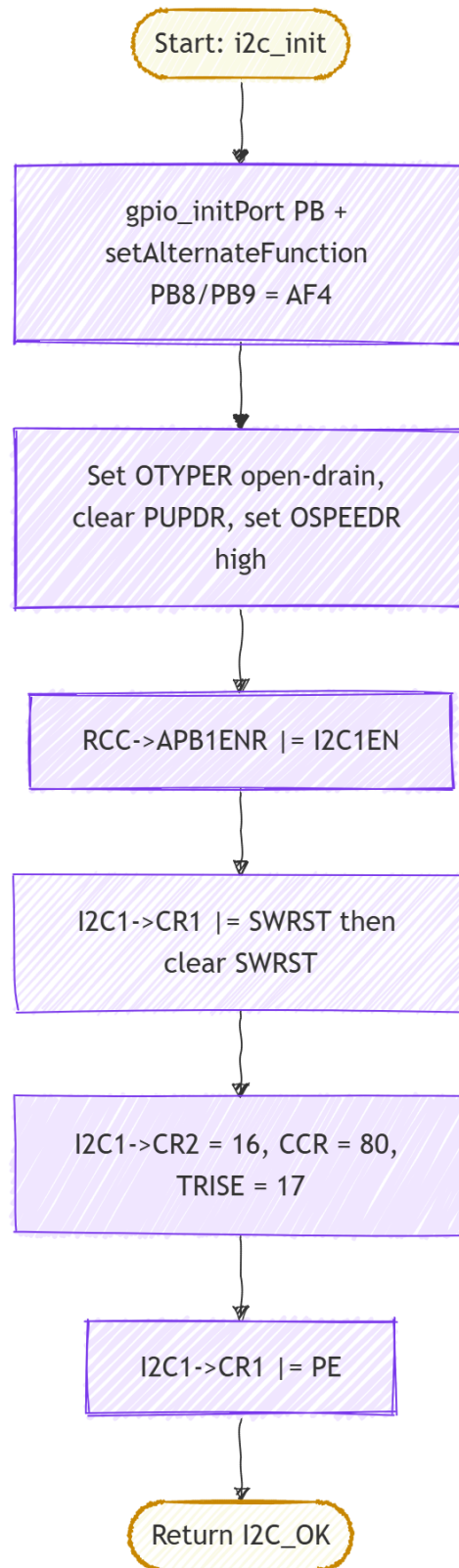


Figure 3. Activity Diagram — i2c_init

Figure — Figure 4. Sequence Diagram — i2c_writeRegDevice

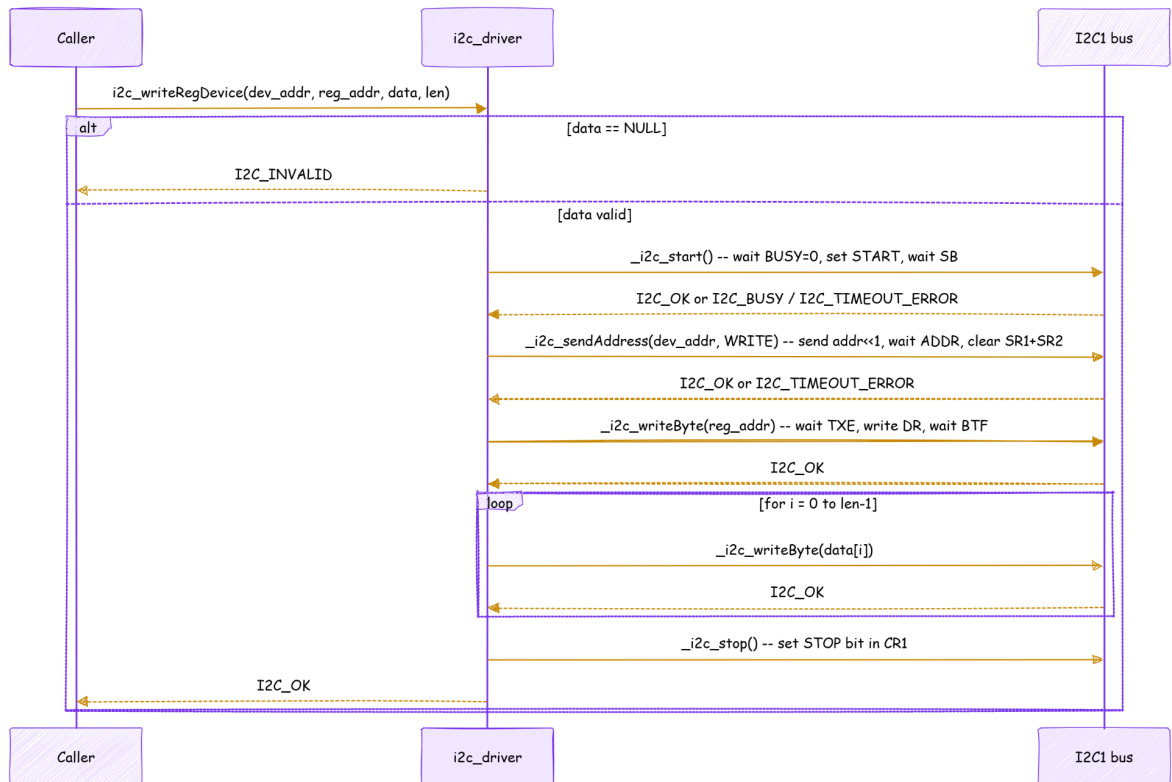


Figure 4. Sequence Diagram — i2c_writeRegDevice

Figure — Figure 5. Activity Diagram — i2c_writeRegDevice

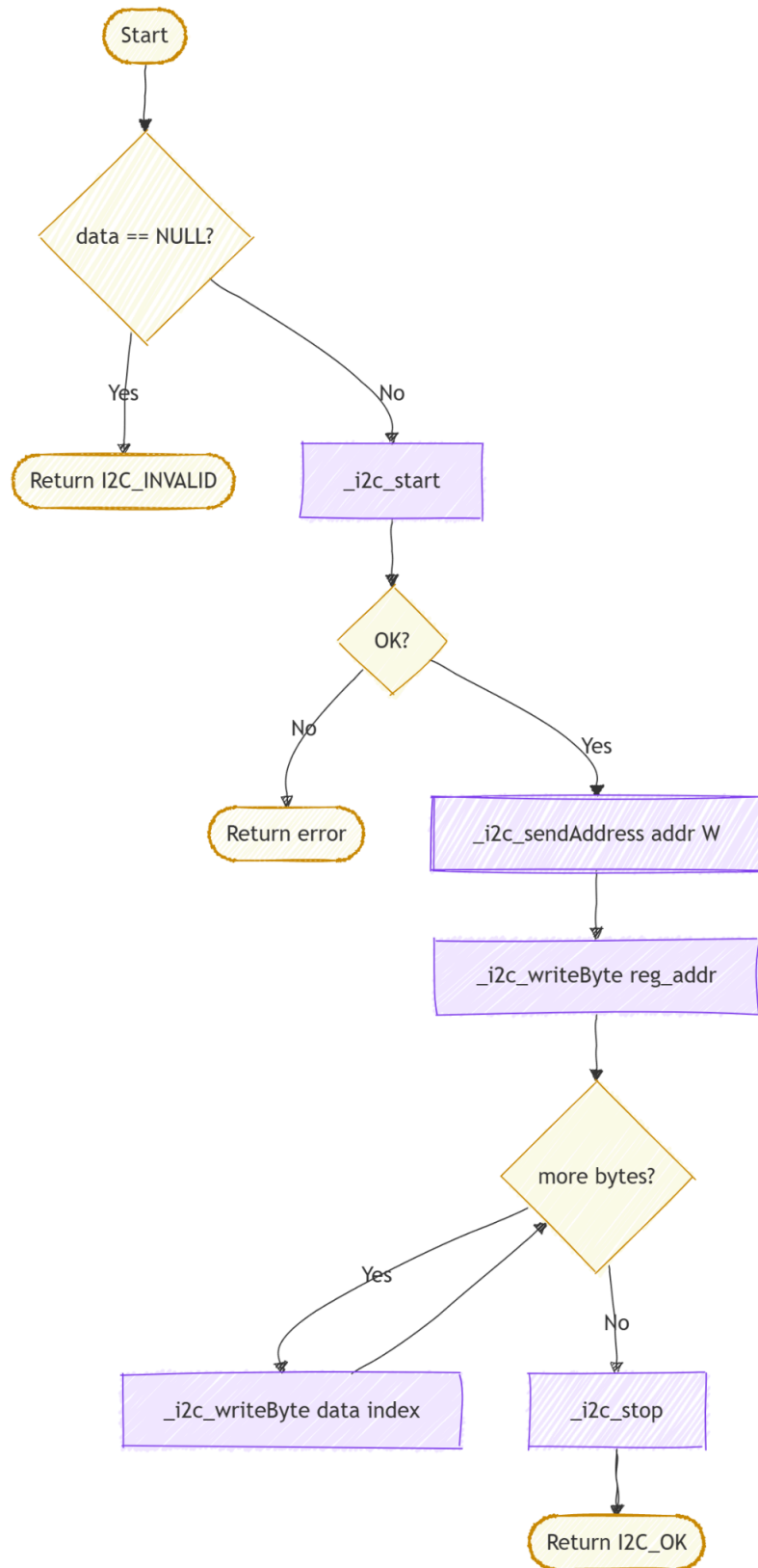


Figure 5. Activity Diagram — i2c_writeRegDevice

Figure — Figure 6. Sequence Diagram — i2c_readRegDevice (two-phase read with repeated START)

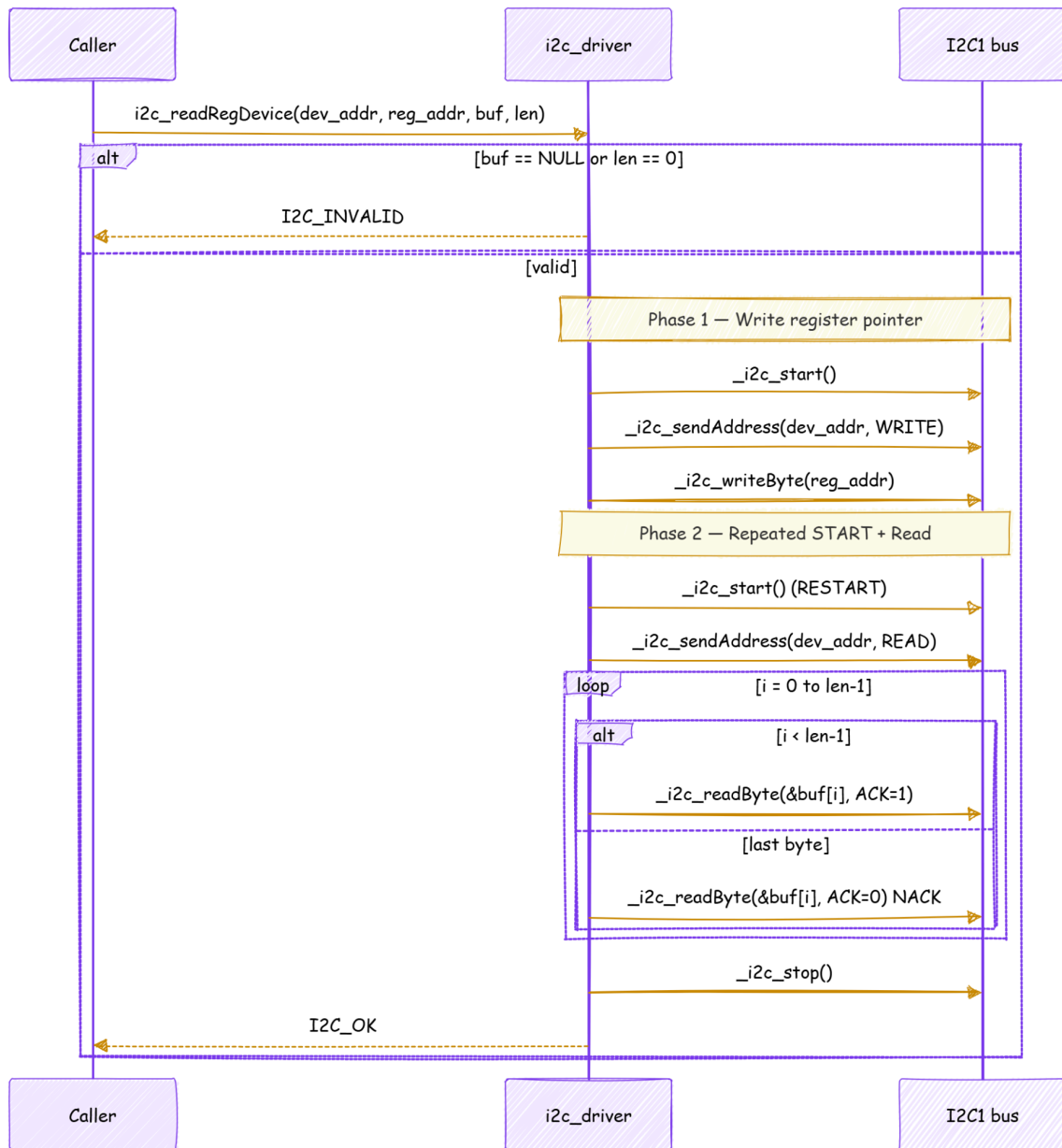


Figure 6. Sequence Diagram — i2c_readRegDevice (two-phase read with repeated START)

Figure — Figure 7. Activity Diagram — i2c_readRegDevice

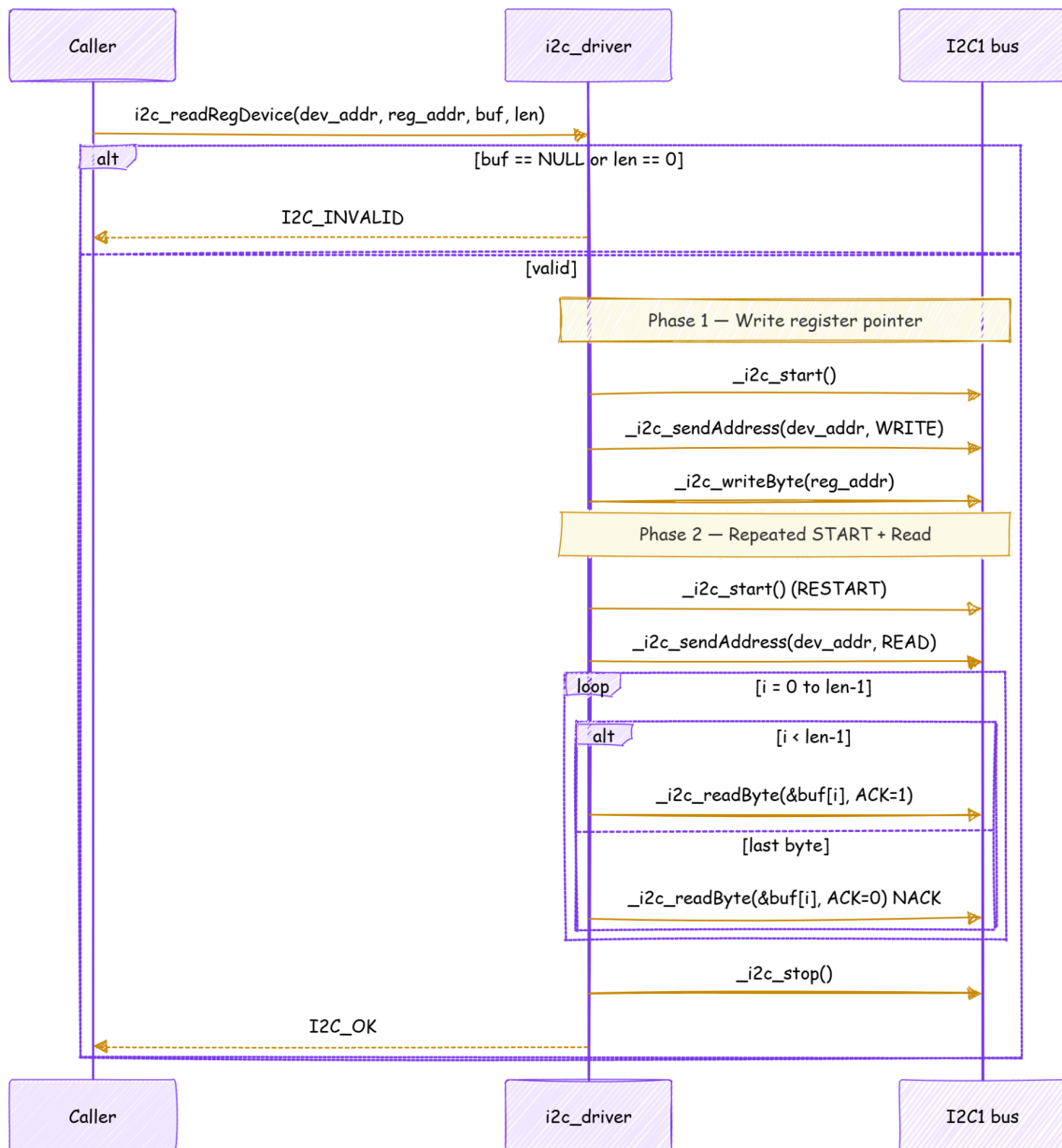


Figure 7. Activity Diagram — i2c_readRegDevice

Figure — Figure 8. State Diagram — I2C Driver

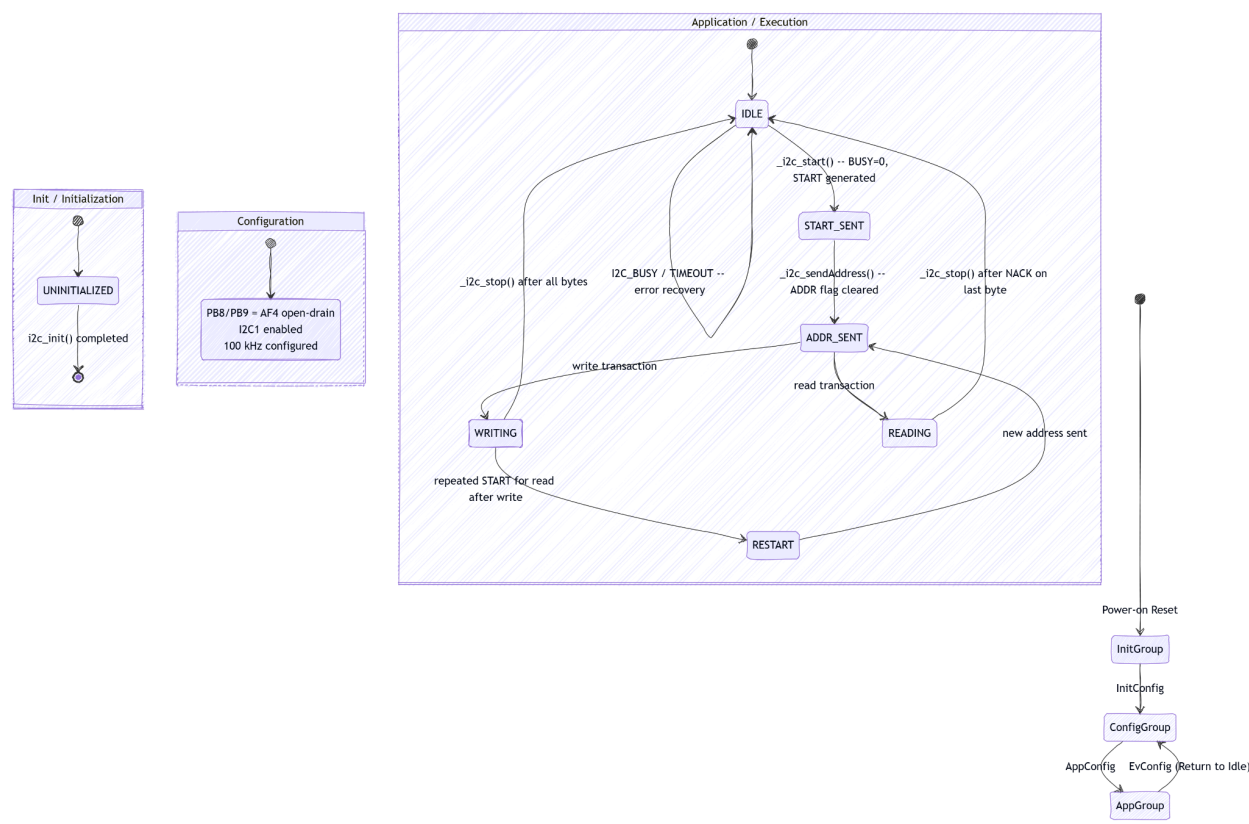


Figure 8. State Diagram — I2C Driver

3.3 I2C Driver

The driver is built around five static helper functions that map directly to individual I2C bus events. This decomposition keeps each public function readable: the complex multi-step bus sequences are expressed as a linear chain of helper calls, each returning an error code that propagates upward. All five public functions share the same error propagation pattern: on any helper failure, `_i2c_stop()` is called before returning the error code to avoid leaving the bus in an undefined state.

Initialization configures PB8 and PB9 as AF4, sets both pins to open-drain mode via `GPIOB->OTYPER` (required by the I2C specification — the bus is a wired-AND topology where any device can pull the line low but none can drive it high), disables pull-ups (`PUPDR = 0`), and sets high speed in `OSPEEDR`. The I2C1 APB1 clock is then enabled, followed by a software reset via `CR1.SWRST` to clear any stale bus state from a previous session. The three timing registers are written in sequence: `CR2.FREQ` field = 16 (APB1 in MHz), `CCR = 80` ($f_{APB1} / (2 \times f_{SCL}) = 16e6 / 200e3$), and `TRISE = 17` (the maximum SCL rise time of 1000 ns expressed in APB1 clock periods plus one). Finally, `CR1.PE` is set to enable the peripheral.

The `_i2c_start()` helper first polls `SR2.BUSY` with a countdown timer to confirm the bus is idle. Once clear, it sets `CR1.START` and waits for `SR1.SB` to confirm that the master has won bus arbitration and the START condition has appeared on the lines. All timeout loops decrement a `uint32_t` counter initialized to `I2C_TIMEOUT = 100000`; if it reaches zero the function returns `I2C_BUSY` or `I2C_TIMEOUT_ERROR`, and the caller issues a STOP to release the bus.

Address transmission requires clearing the SB flag by reading `SR1` before writing the address to `DR` — this is a hardware requirement documented in RM0383. The address is left-shifted by one and OR-ed with the R/W bit: 0 for write, 1 for read. The `ADDR` flag is then cleared by reading `SR1` followed by `SR2` in sequence, as specified by the reference manual.

Write operations poll `TXE` to confirm the data register is empty before each byte write, then wait for `BTF` (Byte Transfer Finished) to confirm the byte has fully shifted out before the next write or STOP. Read operations set or clear `CR1.ACK` before each byte: `ACK=1` tells the peripheral to acknowledge each received byte automatically; `ACK=0` (NACK) is set before the final byte to signal to the slave that no more data is wanted.

In Lab Assignment 05, the typical transaction sequence for `sensor_readData()` is: `i2c_writeRegDevice(0x53, 0x32, ...)` to select register `DATA_X0`, then `i2c_readRegDevice(0x53, 0x32, buf, 6)` to burst-read six bytes. The ADXL345 auto-increments the register pointer, so the six bytes returned are `DATA_X0` through `DATA_Z1`. The sensor layer reassembles the signed 16-bit values as `(buf[1] << 8) | buf[0]` for X, similarly for Y and Z.

4. Application Programming Interfaces

4.1 Data Types and Macros

`i2c_status_t`: the return type for all public I2C driver functions. Five values are defined, two of which are specifically mapped to distinct bus failure modes to aid debugging:

Value	Code / Description
<code>I2C_OK</code>	0 — Operation completed without error.
<code>I2C_ERROR</code>	-1 — Generic bus error not covered by more specific codes.
<code>I2C_BUSY</code>	-2 — Bus was not idle when START was attempted (SR2.BUSY remained set through timeout).
<code>I2C_TIMEOUT_ERROR</code>	-3 — A status flag (SB, ADDR, TXE, BTF, RXNE) did not assert within <code>I2C_TIMEOUT</code> iterations.
<code>I2C_INVALID</code>	-4 — A NULL pointer or a zero-length parameter was passed to a function that requires a valid buffer.

`I2C_TIMEOUT = 100000U`: the decrement-counter limit used in all polling loops. At 16 MHz with no compiler optimization, a tight loop iteration takes approximately one clock cycle; this timeout provides roughly 6 ms of protection before declaring an error. The value may need adjustment if compiler optimization levels change execution timing.

4.2 Error Codes

All public functions return `i2c_status_t`. NULL pointer checks return `I2C_INVALID` immediately, before any bus activity. Bus-level errors (`I2C_BUSY`, `I2C_TIMEOUT_ERROR`) always result in a STOP condition being generated to release the bus before returning to the caller. `I2C_ERROR` is reserved for generic failures and is not currently generated by the implementation.

4.3 Function Definitions

`i2c_init`

Function Name	<code>i2c_init</code>
Syntax	<code>i2c_status_t i2c_init(void)</code>
Parameters (in)	None
Parameters (inout)	None
Parameters (out)	None
Return value	<code>i2c_status_t</code> — <code>I2C_OK</code> always (current implementation assumes 16 MHz HSI).
Description	Configures PB8 (SCL) and PB9 (SDA) as AF4 open-drain, enables I2C1 APB1 clock, performs software reset, sets

	CR2=16, CCR=80, TRISE=17, and enables the peripheral via CR1.PE.
Constraints	Assumes APB1 = 16 MHz (HSI default). External 4.7 kΩ pull-up resistors to 3.3 V are required on PB8 and PB9.
Available via	i2c.h

i2c_writeRegDevice

Function Name	i2c_writeRegDevice
Syntax	i2c_status_t i2c_writeRegDevice(uint8_t dev_addr, uint8_t reg_addr, const uint8_t *data, size_t len)
Parameters (in)	dev_addr — 7-bit device address (not pre-shifted); reg_addr — register address inside the device; data — pointer to byte buffer to transmit; len — number of bytes to write.
Parameters (inout)	None
Parameters (out)	None
Return value	i2c_status_t — I2C_OK on success, I2C_INVALID if data is NULL, I2C_BUSY or I2C_TIMEOUT_ERROR on bus failure.
Description	Executes: START → address+W → reg_addr → data[0..len-1] → STOP. Used by the ADXL345 sensor driver to write configuration registers.
Constraints	Does not validate len == 0 separately; a zero-length transfer issues START and reg_addr then STOP with no data bytes written.
Available via	i2c.h

i2c_writeDevice

Function Name	i2c_writeDevice
Syntax	i2c_status_t i2c_writeDevice(uint8_t dev_addr, const uint8_t *data, size_t len)
Parameters (in)	dev_addr — 7-bit device address; data — pointer to transmit buffer; len — number of bytes to write.
Parameters (inout)	None
Parameters (out)	None

Return value	i2c_status_t — I2C_OK, I2C_INVALID, I2C_BUSY, or I2C_TIMEOUT_ERROR.
Description	Executes: START → address+W → data[0..len-1] → STOP. Does not send a register address; for devices that accept raw byte streams.
Constraints	None.
Available via	i2c.h

i2c_readRegDevice

Function Name	i2c_readRegDevice
Syntax	i2c_status_t i2c_readRegDevice(uint8_t dev_addr, uint8_t reg_addr, uint8_t *buf, size_t len)
Parameters (in)	dev_addr — 7-bit device address; reg_addr — starting register address; len — number of bytes to read.
Parameters (inout)	None
Parameters (out)	buf — uint8_t*: buffer where received bytes are stored.
Return value	i2c_status_t — I2C_OK, I2C_INVALID if buf is NULL or len is 0, I2C_BUSY or I2C_TIMEOUT_ERROR on bus failure.
Description	Two-phase transaction: Phase 1 sends reg_addr (START → addr+W → reg_addr). Phase 2 performs a RESTART and reads len bytes (START → addr+R → buf[0]...buf[len-1] with NACK on last byte → STOP).
Constraints	NACK is asserted before the last byte is read. If len == 1, NACK is set immediately after the ADDR flag is cleared.
Available via	i2c.h

i2c_readDevice

Function Name	i2c_readDevice
Syntax	i2c_status_t i2c_readDevice(uint8_t dev_addr, uint8_t *buf, size_t len)
Parameters (in)	dev_addr — 7-bit device address; len — number of bytes to read.
Parameters (inout)	None
Parameters (out)	buf — uint8_t*: buffer for received data.

Return value	i2c_status_t — I2C_OK, I2C_INVALID if buf is NULL or len is 0, I2C_BUSY or I2C_TIMEOUT_ERROR.
Description	Executes: START → address+R → buf[0..len-1] with NACK on last byte → STOP. No register pointer write; for devices that stream data without an address phase.
Constraints	None.
Available via	i2c.h

4.4 Internal Function Definitions

i2c_start

Function Name	_i2c_start
Syntax	static i2c_status_t _i2c_start(void)
Parameters (in)	None
Parameters (inout)	None
Parameters (out)	None
Return value	i2c_status_t — I2C_OK if START was generated and SB flag confirmed, I2C_BUSY if SR2.BUSY did not clear, I2C_TIMEOUT_ERROR if SB did not set.
Description	Polls SR2.BUSY until clear, sets CR1.START, then waits for SR1.SB. If timeout expires while waiting for SB, generates STOP before returning.
Constraints	Not declared in i2c.h. Internal use only.

i2c_stop

Function Name	_i2c_stop
Syntax	static void _i2c_stop(void)
Parameters (in)	None
Parameters (inout)	None
Parameters (out)	None
Return value	void
Description	Sets CR1.STOP to release the bus. No polling; STOP generation is handled by hardware and does not need confirmation.
Constraints	Not declared in i2c.h.

_i2c_sendAddress

Function Name	<code>_i2c_sendAddress</code>
Syntax	<code>static i2c_status_t _i2c_sendAddress(uint8_t address, uint8_t read)</code>
Parameters (in)	address — 7-bit device address; read — 1 for read direction, 0 for write direction.
Parameters (inout)	None
Parameters (out)	None
Return value	<code>i2c_status_t</code> — <code>I2C_OK</code> if ADDR flag asserted, <code>I2C_TIMEOUT_ERROR</code> if ADDR did not set within timeout.
Description	Reads SR1 to clear SB (hardware requirement), writes (address << 1) read to DR, waits for SR1.ADDR, then clears ADDR by reading SR1 then SR2 in sequence. Clears SR1.AF on timeout.
Constraints	Not declared in <code>i2c.h</code> .

_i2c_writeByte

Function Name	<code>_i2c_writeByte</code>
Syntax	<code>static i2c_status_t _i2c_writeByte(uint8_t data)</code>
Parameters (in)	data — byte to transmit.
Parameters (inout)	None
Parameters (out)	None
Return value	<code>i2c_status_t</code> — <code>I2C_OK</code> , <code>I2C_TIMEOUT_ERROR</code> if TXE or BTF did not assert within timeout.
Description	Polls SR1.TXE until set, writes data to DR, then polls SR1.BTF until set to confirm the shift register has fully transmitted the byte.
Constraints	Not declared in <code>i2c.h</code> .

_i2c_readByte

Function Name	<code>_i2c_readByte</code>
Syntax	<code>static i2c_status_t _i2c_readByte(uint8_t *data, uint8_t ack)</code>

Parameters (in)	data — pointer to store received byte; ack — 1 = send ACK (more bytes), 0 = send NACK (last byte).
Parameters (inout)	None
Parameters (out)	None
Return value	i2c_status_t — I2C_OK, I2C_TIMEOUT_ERROR if RXNE did not set.
Description	Sets or clears CR1.ACK based on ack argument, polls SR1.RXNE until set, then reads DR into *data.
Constraints	Caller must set ack=0 before the last byte to generate NACK. Not declared in i2c.h.

5. Future Enhancements

- Interrupt-driven transfers: replace all polling loops with I2C event interrupts (I2C1_EV_IRQn) and a state machine, allowing the CPU to perform other work during multi-byte transfers.
- DMA support: connect I2C1 TX/RX to DMA1 streams to transfer large buffers without CPU intervention — relevant for reading multi-axis FIFO data or updating display drivers.
- Multi-master arbitration: add bus arbitration detection via SR1.ARLO and implement retry logic for shared-bus environments.
- Fast mode (400 kHz): expose a speed parameter in i2c_init() and recalculate CCR and TRISE accordingly (CCR = 13 for 400 kHz @ 16 MHz in standard CCR mode).
- SMBUS support: extend CR1 configuration to enable the SMBus mode (CR1.SMBUS), adding host and device mode compliance for PMBus power management applications.
- 10-bit addressing: add an i2c_writeDevice10 and i2c_readDevice10 variant that sets CR2.ADD10 and transmits the two-byte address sequence required for 10-bit I2C devices.

6. References

- [1] IEEE Std 1016-2009, "IEEE Standard for Information Technology -- Systems Design -- Software Design Descriptions," IEEE, New York, NY, 2009.
- [2] STMicroelectronics, "RM0383: STM32F411xC/E Reference Manual," Rev. 3, 2016. Available:
https://www.st.com/resource/en/reference_manual/rm0383-stm32f411xce-advanced-armbased-32bit-mcus-stmicroelectronics.pdf
- [3] STMicroelectronics, "UM1724: STM32 Nucleo-64 Boards User Manual," Rev. 14, 2021.
- [4] Analog Devices, "ADXL345 Digital Accelerometer Datasheet," Rev. H, 2015. Available:
<https://www.analog.com/media/en/technical-documentation/data-sheets/ADXL345.pdf>
- [5] Gbati, O., "Embedded Systems Programming with the STM32 Microcontroller," Independently published, 2023.